Here are the most important things about Spring Framework:

1. **Inversion of Control (IoC)**: Spring's core principle is IoC, which decouples the execution of a task from the implementation. It manages object creation and dependencies, enabling loose coupling and easier testing.
2. **Dependency Injection (DI)**: Spring facilitates DI, where the framework injects dependencies into objects at runtime. This reduces boilerplate code and enhances modularity.
3. **Aspect-Oriented Programming (AOP)**: Spring provides AOP support, allowing cross-cutting concerns (like logging, security, transactions) to be handled separately from the main business logic.
4. **Spring MVC**: A web framework that follows the Model-View-Controller design pattern. It simplifies creating web applications by managing HTTP requests, responses, and model binding.
5. **Spring Data**: Provides tools to simplify data access and manipulation with databases, including JPA, MongoDB, and more. Spring Data offers repository abstractions for database interactions.
6. **Spring Security**: A comprehensive solution for securing Java applications. It handles authentication, authorization, and common security vulnerabilities like CSRF.
7. **Spring Boot**: A project built on Spring that simplifies configuring and running Spring applications by offering defaults, auto-configuration, and embedded servers.
8. **Transaction Management**: Spring provides declarative transaction management, allowing developers to manage transactions with annotations like `@Transactional`, making database operations consistent and reliable.
9. **Spring Cloud**: Designed for microservice architecture, Spring Cloud provides tools for distributed systems, such as configuration management, service discovery, load balancing, and fault tolerance.

In Spring, Inversion of Control (IoC) is primarily implemented using **Dependency Injection (DI)**. Here's how to use IoC in Spring:

1. Define Beans

In Spring, a *bean* is an object that is managed by the Spring IoC container. You can define beans either through annotations or XML configuration.

Using Annotations:

You mark a class as a bean using the `@Component`, `@Service`, `@Repository`, or `@Controller` annotations.

java
Skopiuj kod
```
@Component
public class MyService {
```

```
    public void doSomething() {
        System.out.println("Doing something...");
    }
}
```

2. Inject Dependencies

To inject dependencies, you can use the `@Autowired` annotation or constructor-based injection.

Field Injection (via `@Autowired`):

java
Skopiuj kod

```
@Component
public class MyController {

    @Autowired
    private MyService myService;  // Spring injects MyService bean
here

    public void execute() {
        myService.doSomething();
    }
}
```

Constructor Injection:

java
Skopiuj kod

```
@Component
public class MyController {
    private final MyService myService;

    @Autowired
    public MyController(MyService myService) {
        this.myService = myService;
    }

    public void execute() {
        myService.doSomething();
    }
}
```

3. Configure Component Scanning

Spring needs to know where to look for the beans. You configure this by enabling component scanning.

Java-based Configuration:
java
Skopiuj kod
```
@Configuration
@ComponentScan("com.example")  // specify the package to scan
public class AppConfig {
}
```

XML Configuration:
xml
Skopiuj kod
```
<context:component-scan base-package="com.example" />
```

4. Load the Spring Application Context

The Spring IoC container is initialized through the `ApplicationContext`, which reads the configuration, manages beans, and injects dependencies.

java
Skopiuj kod
```
public class MainApp {
    public static void main(String[] args) {
        ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
        MyController controller =
context.getBean(MyController.class);
        controller.execute();  // This will call
MyService.doSomething()
    }
}
```

Summary:

- Define your beans with `@Component` or other stereotype annotations.
- Use `@Autowired` or constructor injection to inject dependencies.

- Set up component scanning.
- Use `ApplicationContext` to load and manage beans.

This is how Spring's IoC container manages dependencies and creates loosely coupled, testable components.

In Spring, the component (or bean) lifecycle is the process that a Spring-managed bean undergoes from creation to destruction within the Spring IoC container. The lifecycle includes several key phases that are managed by the Spring framework. Here's an overview of the main stages:

1. Bean Instantiation

- **What Happens**: The Spring container creates an instance of the bean by calling its constructor.
- **Triggered By**: The container when the bean is requested for the first time (if using lazy initialization) or during container startup (if using eager initialization, which is the default for singleton beans).

2. Dependency Injection

- **What Happens**: After instantiation, Spring performs dependency injection, either through constructor, setter, or field injection, based on how the bean has been configured.

3. Bean Name Assignment

- **What Happens**: If the bean is defined in XML or Java configuration, the Spring container assigns a name (or ID) to the bean.

4. Bean Initialization (Optional Hooks)

- **What Happens**: This is when the bean is initialized and given a chance to perform any custom setup or initialization logic.
- **Methods Involved**:
  - **`@PostConstruct`**: Method annotated with `@PostConstruct` will be executed after the dependencies are injected.
  - **`InitializingBean` interface**: Implementing the `afterPropertiesSet()` method of this interface allows you to define initialization logic.
  - **Custom Init Method**: You can also define a custom init method using the `init-method` attribute in XML or `@Bean(initMethod = "init")` in Java configuration.

java
Skopiuj kod
```
@Component
```

```
public class MyBean implements InitializingBean {
    @Override
    public void afterPropertiesSet() throws Exception {
        // Custom initialization logic
        System.out.println("Bean is initialized!");
    }
}
```

Or with `@PostConstruct`:

java
Skopiuj kod
```
@Component
public class MyBean {
    @PostConstruct
    public void init() {
        // Custom initialization logic
        System.out.println("Bean is initialized!");
    }
}
```

5. Bean Usage

- **What Happens**: The bean is fully initialized and ready for use. It can be used to handle business logic or perform its designed function in the application.
- **How It Works**: During this phase, the Spring container manages the bean's lifecycle as part of the application context, handling its dependencies and ensuring its availability.

6. Bean Destruction (Optional Hooks)

- **What Happens**: When the Spring IoC container is shut down, the beans are destroyed, and Spring provides hooks to release resources or perform cleanup operations before the bean is destroyed.
- **Methods Involved**:
    - **`@PreDestroy`**: A method annotated with `@PreDestroy` will be executed just before the bean is destroyed.
    - **`DisposableBean` interface**: Implementing the `destroy()` method allows custom cleanup before the bean is destroyed.
    - **Custom Destroy Method**: You can also define a custom destroy method using the `destroy-method` attribute in XML or `@Bean(destroyMethod = "destroy")` in Java configuration.

java
Skopiuj kod
```java
@Component
public class MyBean implements DisposableBean {
    @Override
    public void destroy() throws Exception {
        // Custom cleanup logic
        System.out.println("Bean is being destroyed!");
    }
}
```

Or with `@PreDestroy`:

java
Skopiuj kod
```java
@Component
public class MyBean {
    @PreDestroy
    public void cleanup() {
        // Custom cleanup logic
        System.out.println("Bean is being destroyed!");
    }
}
```

7. Bean Garbage Collection

- **What Happens**: After the bean is destroyed, it is eligible for garbage collection by the Java Virtual Machine (JVM), and Spring no longer manages it.

Summary of Spring Bean Lifecycle:

1. **Instantiation**: Bean is created.
2. **Dependency Injection**: Dependencies are injected.
3. **Initialization**: Custom initialization logic (via `@PostConstruct`, `afterPropertiesSet()`, or custom init methods).
4. **Usage**: The bean is ready for use in the application.
5. **Destruction**: Cleanup (via `@PreDestroy`, `destroy()`, or custom destroy methods).
6. **Garbage Collection**: The JVM reclaims the memory after destruction.

Spring provides flexibility with multiple ways to hook into the lifecycle for custom initialization and destruction behavior.

A **cyclic dependency** in Spring (or in general) occurs when two or more beans depend on each other, either directly or indirectly, creating a circular reference. This means that Bean A depends on Bean B, while Bean B also depends on Bean A, either directly or through other beans, leading to a cycle that Spring cannot resolve.

Example of Cyclic Dependency:

java
Skopiuj kod
```
@Component
public class A {
    @Autowired
    private B b;
}

@Component
public class B {
    @Autowired
    private A a;
}
```

In this case:

- `A` requires an instance of `B` to be injected.
- `B` requires an instance of `A` to be injected.

Spring's IoC container will not be able to resolve this cycle, as it would try to create `A`, but to do so, it needs `B`, which in turn needs `A`, leading to an infinite loop. This results in a **`BeanCurrentlyInCreationException`**.

Types of Cyclic Dependencies

1. **Constructor Injection Cyclic Dependency**:
    - With constructor injection, Spring attempts to create both beans at the same time, making it impossible to resolve the cycle. This results in an immediate failure.

Example:
java
Skopiuj kod
```
@Component
public class A {
    private final B b;

    @Autowired
```

```
    public A(B b) {
        this.b = b;
    }
}

@Component
public class B {
    private final A a;

    @Autowired
    public B(A a) {
        this.a = a;
    }
}
```

2.
3. **Setter/Field Injection Cyclic Dependency**:
    - With setter or field injection, Spring may initially attempt to resolve the cycle through a technique called **partial injection**. It creates one of the beans first and then injects dependencies later. This is sometimes allowed by Spring, but it's generally considered bad practice and can lead to subtle issues.

How to Solve Cyclic Dependency

1. **Refactor the Code**:
    - The best solution is to **break the cyclic dependency** by refactoring the code so that beans no longer depend on each other directly.
2. Example:
    - Introduce a third service or mediator that both `A` and `B` depend on, which removes the direct dependency between them.
3. **Use `@Lazy` Annotation**:
    - You can use the `@Lazy` annotation on one of the dependencies to delay the injection of a bean until it's actually needed, which can sometimes resolve the issue.

Example:
java
Skopiuj kod

```
@Component
public class A {
    @Autowired
    @Lazy
    private B b;
}
```

```
@Component
public class B {
    @Autowired
    private A a;
}
```

4. In this case, Spring will inject a proxy for `B` and resolve it only when it is actually needed, avoiding the immediate circular reference during bean creation.
5. **Use Setter Injection Instead of Constructor Injection**:
    - With setter injection, Spring can handle some cyclic dependencies by creating one of the beans first, then injecting the other later.

Example:
java
Skopiuj kod

```
@Component
public class A {
    private B b;

    @Autowired
    public void setB(B b) {
        this.b = b;
    }
}

@Component
public class B {
    private A a;

    @Autowired
    public void setA(A a) {
        this.a = a;
    }
}
```

6.
7. **Use `@PostConstruct` for Delayed Initialization**:
    - You can use `@PostConstruct` to initialize a dependency after Spring has fully created the beans, thus breaking the cycle.

Conclusion:

Cyclic dependencies are typically a design flaw and should be avoided. Refactoring the design, using `@Lazy`, or switching to setter injection are ways to mitigate them, but it's generally best to refactor your code to avoid circular references entirely.

When there are multiple implementations of the same interface in Spring, the IoC container can get confused about which implementation to inject, leading to issues like:

- **`NoUniqueBeanDefinitionException`**: This occurs when Spring finds more than one candidate bean for a particular type and doesn't know which one to inject.

Here's why and how this happens, and how to fix it.

When Multiple Implementations of the Same Interface Happen

This typically occurs when:

1. **Multiple Beans Implement the Same Interface**:
    - You define multiple beans that implement the same interface, and Spring doesn't know which one to inject.

Example:
java
Skopiuj kod
```
public interface Service {
    void doWork();
}

@Component
public class ServiceImpl1 implements Service {
    public void doWork() {
        System.out.println("ServiceImpl1");
    }
}

@Component
public class ServiceImpl2 implements Service {
    public void doWork() {
        System.out.println("ServiceImpl2");
    }
}
```

2. If you attempt to inject the `Service` interface somewhere, Spring won't know whether to inject `ServiceImpl1` or `ServiceImpl2`.

3. **Third-Party Libraries**:
    - Some third-party libraries may define beans that implement the same interface, causing conflicts if you define custom implementations in your application.
4. **Configuration Classes**:
    - If multiple beans of the same type are defined in a Spring configuration class, the ambiguity could also arise.

Solutions to Resolve Multiple Implementations of the Same Interface

1. Use `@Primary` Annotation

- Mark one of the beans as the default (or primary) bean by using the `@Primary` annotation. This tells Spring to inject this bean when no specific implementation is provided.

Example:

java
Skopiuj kod
```java
@Component
@Primary
public class ServiceImpl1 implements Service {
    public void doWork() {
        System.out.println("ServiceImpl1");
    }
}

@Component
public class ServiceImpl2 implements Service {
    public void doWork() {
        System.out.println("ServiceImpl2");
    }
}
```

Now, when you inject the `Service` interface, Spring will default to `ServiceImpl1` because of the `@Primary` annotation.

2. Use `@Qualifier` Annotation

- The `@Qualifier` annotation allows you to explicitly specify which bean to inject when there are multiple implementations. You can use the bean name or a custom qualifier to disambiguate.

Example:

java
Skopiuj kod
```java
@Component("serviceImpl1")
public class ServiceImpl1 implements Service {
    public void doWork() {
        System.out.println("ServiceImpl1");
    }
}

@Component("serviceImpl2")
public class ServiceImpl2 implements Service {
    public void doWork() {
        System.out.println("ServiceImpl2");
    }
}

@Component
public class MyController {
    private final Service service;

    @Autowired
    public MyController(@Qualifier("serviceImpl1") Service service)
{
        this.service = service;
    }

    public void execute() {
        service.doWork();  // This will call ServiceImpl1
    }
}
```

3. Use `@Bean` Method Names in Configuration

- If you're using Java configuration with `@Bean` methods, the method name becomes the bean name. You can use this method name in the `@Qualifier` annotation to specify which bean to inject.

Example:

java
Skopiuj kod
```java
@Configuration
```

```
public class AppConfig {

    @Bean
    public Service serviceImpl1() {
        return new ServiceImpl1();
    }

    @Bean
    public Service serviceImpl2() {
        return new ServiceImpl2();
    }
}

@Component
public class MyController {
    private final Service service;

    @Autowired
    public MyController(@Qualifier("serviceImpl1") Service service)
{
        this.service = service;
    }
}
```

4. Inject All Implementations with `List` or `Map`

- You can inject all implementations of a given interface as a `List` or `Map` of beans. This is useful if you want to dynamically choose between the different implementations at runtime.

Example:

java
Skopiuj kod

```
@Component
public class MyController {

    private final List<Service> services;

    @Autowired
    public MyController(List<Service> services) {
        this.services = services;
```

```
    }

    public void executeAll() {
        for (Service service : services) {
            service.doWork();
        }
    }
}
```

Spring will inject all `Service` implementations into the `services` list, allowing you to iterate through and use them all.

5. Use Custom Qualifiers

- If you have more complex scenarios, you can define custom qualifiers for your beans to further specify which implementation should be injected.

Example:

java
Skopiuj kod
```
@Component
@Qualifier("importantService")
public class ImportantServiceImpl implements Service {
    public void doWork() {
        System.out.println("ImportantService");
    }
}

@Component
@Qualifier("normalService")
public class NormalServiceImpl implements Service {
    public void doWork() {
        System.out.println("NormalService");
    }
}

@Component
public class MyController {
    private final Service service;

    @Autowired
```

```
    public MyController(@Qualifier("importantService") Service
service) {
        this.service = service;
    }
}
```

Summary of Solutions

1. `@Primary`: Mark one bean as the primary/default option.
2. `@Qualifier`: Explicitly specify the bean by name or qualifier.
3. `@Bean` **Method Names**: Use method names in configuration classes to define beans and refer to them by name.
4. **Inject All Implementations**: Inject a `List` or `Map` to work with all implementations.
5. **Custom Qualifiers**: Create custom qualifiers for more specific control over injection.

Each of these methods helps Spring resolve which implementation to inject when multiple beans of the same type exist in the application context.

A Spring configuration file is used to define beans and dependencies within a Spring application. It can either be an XML file or a class annotated with `@Configuration` in Java. Here's an overview of both:

1. XML-based Configuration

In XML-based configuration, beans are declared and wired together using XML tags inside an XML file (e.g., `applicationContext.xml`).

Example:

xml
Skopiuj kod
```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://www.springframework.org/schema/beans

http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="myBean" class="com.example.MyBean">
        <property name="name" value="Spring Bean Example"/>
    </bean>
```

```
</beans>
```

2. Java-based Configuration

Java configuration uses classes annotated with `@Configuration` and `@Bean` annotations to define beans programmatically.

Example:

java
Skopiuj kod
```
@Configuration
public class AppConfig {

    @Bean
    public MyBean myBean() {
        return new MyBean("Spring Bean Example");
    }
}
```

Both methods achieve the same goal: managing and wiring beans for dependency injection.

4o

Powiedziałeś(-aś):
Describe Spring bean scopes

ChatGPT powiedział:
ChatGPT

Spring bean scopes define the lifecycle and visibility of beans within the Spring container. There are five primary bean scopes in Spring:

1. Singleton (Default)

- **Description**: Only **one instance** of the bean is created for the entire Spring IoC container, and this instance is shared across all requests and references.
- **Usage**: It is the default scope if not specified. It's typically used for stateless beans.

Example:
java
Skopiuj kod
```
@Bean
@Scope("singleton") // or omit this as it's the default
public MyBean myBean() {
```

```
    return new MyBean();
}
```

-

2. Prototype

- **Description**: A **new instance** of the bean is created **every time** it is requested from the container.
- **Usage**: Useful for stateful beans or when the bean's state changes frequently.

Example:
java
Skopiuj kod
```
@Bean
@Scope("prototype")
public MyBean myBean() {
    return new MyBean();
}
```

-

3. Request (Web Applications)

- **Description**: A new bean instance is created for each **HTTP request** and is only available during the lifespan of that request.
- **Usage**: Typically used in web applications to handle request-specific data.

Example:
java
Skopiuj kod
```
@Bean
@Scope("request")
public MyBean myBean() {
    return new MyBean();
}
```

-

4. Session (Web Applications)

- **Description**: A bean is created once per **HTTP session** and is shared across the session.
- **Usage**: Used for session-scoped data, such as session-specific user information.

Example:
java

Skopiuj kod
```
@Bean
@Scope("session")
public MyBean myBean() {
    return new MyBean();
}
```

-

5. Application (Web Applications)

- **Description**: A single instance of the bean is created for the entire **ServletContext** (web application context), and it is shared across all sessions and requests.
- **Usage**: Used when you need to share the bean across the entire application.

Example:
java
Skopiuj kod
```
@Bean
@Scope("application")
public MyBean myBean() {
    return new MyBean();
}
```

-

These scopes help manage bean lifecycles based on the requirements of the application and its context.

A Spring configuration file is used to define beans and dependencies within a Spring application. It can either be an XML file or a class annotated with `@Configuration` in Java. Here's an overview of both:

1. XML-based Configuration

In XML-based configuration, beans are declared and wired together using XML tags inside an XML file (e.g., `applicationContext.xml`).

Example:

xml
Skopiuj kod
```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="myBean" class="com.example.MyBean">
        <property name="name" value="Spring Bean Example"/>
    </bean>

</beans>
```

2. Java-based Configuration

Java configuration uses classes annotated with `@Configuration` and `@Bean` annotations to define beans programmatically.

Example:

java
Skopiuj kod
```
@Configuration
public class AppConfig {

    @Bean
    public MyBean myBean() {
        return new MyBean("Spring Bean Example");
    }
}
```

Both methods achieve the same goal: managing and wiring beans for dependency injection.

4o

Powiedziałeś(-aś):
Describe Spring bean scopes

ChatGPT powiedział:
ChatGPT

Spring bean scopes define the lifecycle and visibility of beans within the Spring container. There are five primary bean scopes in Spring:

1. Singleton (Default)

- **Description**: Only **one instance** of the bean is created for the entire Spring IoC container, and this instance is shared across all requests and references.
- **Usage**: It is the default scope if not specified. It's typically used for stateless beans.

Example:
java
Skopiuj kod
```
@Bean
@Scope("singleton") // or omit this as it's the default
public MyBean myBean() {
    return new MyBean();
}
```

-

2. Prototype

- **Description**: A **new instance** of the bean is created **every time** it is requested from the container.
- **Usage**: Useful for stateful beans or when the bean's state changes frequently.

Example:
java
Skopiuj kod
```
@Bean
@Scope("prototype")
public MyBean myBean() {
    return new MyBean();
}
```

-

3. Request (Web Applications)

- **Description**: A new bean instance is created for each **HTTP request** and is only available during the lifespan of that request.
- **Usage**: Typically used in web applications to handle request-specific data.

Example:
java
Skopiuj kod
```
@Bean
@Scope("request")
public MyBean myBean() {
    return new MyBean();
}
```

-

4. Session (Web Applications)

- **Description**: A bean is created once per **HTTP session** and is shared across the session.
- **Usage**: Used for session-scoped data, such as session-specific user information.

Example:
java
Skopiuj kod
```
@Bean
@Scope("session")
public MyBean myBean() {
    return new MyBean();
}
```

-

5. Application (Web Applications)

- **Description**: A single instance of the bean is created for the entire **ServletContext** (web application context), and it is shared across all sessions and requests.
- **Usage**: Used when you need to share the bean across the entire application.

Example:
java
Skopiuj kod
```
@Bean
@Scope("application")
public MyBean myBean() {
    return new MyBean();
}
```

-

These scopes help manage bean lifecycles based on the requirements of the application and its context.

4o

Powiedziałeś(-aś):
How Spring support tests

ChatGPT powiedział:
ChatGPT

Spring provides robust support for testing, allowing developers to test various components of a Spring application in isolation or in integration with the Spring context. Key features and support include:

1. Spring TestContext Framework

Spring's `TestContext` framework is the core of its testing support, enabling developers to create integration tests that are managed by the Spring container. Key annotations and classes:

- **`@RunWith(SpringRunner.class)`**: Integrates the test with the Spring TestContext framework, typically used with JUnit.
- **`@SpringBootTest`**: Loads the full application context for integration tests.
- **`@ContextConfiguration`**: Specifies the configuration files or classes to use for setting up the context.
- **`@TestConfiguration`**: Used to define additional beans for testing purposes without affecting the main application context.

2. Dependency Injection in Tests

Spring can inject dependencies directly into your test classes just like in the application, supporting both `@Autowired` and `@MockBean`.

- **`@Autowired`**: Injects real beans into your test class.
- **`@MockBean`**: Replaces a bean in the Spring context with a Mockito mock, useful for isolating tests.

Example:

java
Skopiuj kod
```
@RunWith(SpringRunner.class)
@SpringBootTest
public class MyServiceTest {

    @Autowired
    private MyService myService;

    @MockBean
    private MyRepository myRepository;

    @Test
    public void testServiceMethod() {
        when(myRepository.findSomething()).thenReturn(new
Something());
        Something result = myService.performAction();
        assertNotNull(result);
    }
}
```

3. Transactional Tests

- **`@Transactional`**: By annotating a test class or method with `@Transactional`, tests can run within a transaction. After the test completes, the transaction is automatically rolled back, ensuring a clean state for each test.

4. Mocking Support (Mockito and Spring)

Spring integrates well with **Mockito**, allowing you to mock dependencies during tests, enabling unit tests without the need for a full application context.

- **`@MockBean`**: A Spring-specific integration of Mockito, replacing specific beans in the test context with mocks.

5. Web Layer Testing

Spring provides specific support for testing the web layer without starting a full web server.

`@WebMvcTest`: Used for testing Spring MVC controllers without loading the entire application context. It only loads the web layer components like controllers, filters, etc.
Example:
java
Skopiuj kod

```
@WebMvcTest(MyController.class)
public class MyControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    public void testController() throws Exception {
        mockMvc.perform(get("/my-endpoint"))
                .andExpect(status().isOk())
                .andExpect(content().string("Expected Result"));
    }
}
```

-

6. Data Access Layer Testing

Spring provides the `@DataJpaTest` annotation for testing repositories and the JPA layer. It configures in-memory databases, auto-configures Hibernate, and provides everything necessary to test the data access layer.

Example:

java
Skopiuj kod
```java
@RunWith(SpringRunner.class)
@DataJpaTest
public class MyRepositoryTest {

    @Autowired
    private MyRepository myRepository;

    @Test
    public void testFindById() {
        MyEntity entity = myRepository.findById(1L).orElse(null);
        assertNotNull(entity);
    }
}
```

7. Utilities for Testing

- **`TestRestTemplate`**: Provides easy-to-use HTTP client methods for testing REST APIs in Spring Boot applications.
- **`MockMvc`**: Allows for testing the web layer by mocking HTTP requests and responses, simulating actual interactions with controllers.

Spring's testing framework allows developers to thoroughly test their applications, ensuring reliable and maintainable code, whether you're unit testing components or writing full integration tests.